MUSIC RECOMMENDATION SYSTEM

20.09.2026

Presented by:

SWAMEENI MORE- 56

PAYAL JAIN- 51

SHREYA SURVE- 07

HARCHITA JAIN- 42

The project started with a large music listening dataset. The overall goal was to organize that data properly and then use it to build a music recommendation system.

Person 1 — Data Collection and Cleaning

Person 1 worked on preparing the original dataset so the rest of the team could use it.

They:

- collected the music and listening data,
- checked for missing values,
- removed duplicate or unnecessary records,
- cleaned the columns and data types,
- prepared the audio-feature information needed later.

Output:

A clean dataset that could be passed to the next part of the project.

Person 2 — Data Warehousing

The cleaned data was organized into fact and dimension-style tables so that it could be used more easily for analysis and recommendation later.

The main tables created were:

Fact_Listening

Contains the main listening-related records and audio information.

Dim_User

Contains user-related information.

Dim_Song

Contains song-related information.

Dim_Audio_Features

Contains information such as:

- tempo
- energy
- danceability
- acousticness
- instrumentalness
- liveness
- valence
- loudness
- speechiness

The purpose was simply to make the data **more organized and easier to work with**.

Output:
A structured set of fact and dimension tables exported as CSV files, which became the input for the recommendation part.

Person 3 — Music Recommendation

Person 3 used the organized data to find songs that are similar to each other.

They used the audio features of songs and a **KNN/similarity approach** to find nearby or similar songs.

They tested different K values:

K = 3, 5, 7, 9, 11

For each K value, recommendation outputs were generated.

Output:
The system looks at a song's characteristics and finds other songs that have similar characteristics.

Person 4 — Recommendation Evaluation

Person 4 took the recommendation outputs for **K = 3, 5, 7, 9 and 11** and evaluated them.

Combined the recommendation results with the music data using **Joiner** nodes.

Used a **Rule Engine** to create `relevant_flag` based on the `liked` value.

Used **GroupBy** to calculate:

- `Sum(relevant_flag)` → number of relevant recommendations
- `Mean(distance)` → average recommendation distance

Combined the five K results using **Concatenate**.

Created a separate **diversity-analysis branch** using the nine audio features: `tempo, energy, danceability, acousticness, instrumentalness, liveness, valence, loudness, speechiness`.

Calculated standard deviation and then an average for the diversity analysis.

Combined the diversity results for the different K values.

Output

K	Relevant Recommendations	Mean Distance
3	2	0.009
5	4	0.014
7	5	0.015
9	5	0.016
11	5	0.015

How Mean Distance Helps Find Similar Songs

For each recommended song, the KNN system calculates a **distance** between the reference song and the other songs using their audio features such as energy, danceability, tempo, and valence.

A **smaller distance means the songs are more similar** based on those features.

K is the number of nearest songs we want the KNN system to consider as recommendations. So when K = 3, the system finds the three songs with the smallest distance from the reference song.

Distance tells us how similar two songs are based on their audio features. Smaller distance means more similar.

Then why test K = 3, 5, 7, 9, 11?

Because we want to see **how many nearest songs should be considered**.

- **K = 3:** only the 3 closest songs
- **K = 5:** the 5 closest songs
- **K = 7:** the 7 closest songs
- and so on.

As K increases, we include more songs, but the additional songs can be farther away.

That is why you may see mean distance change.

Recommendation System: prototype